

Risk Level: NEEDS REVIEW

62% safe

The function is short and likely works for common inputs, but it silently assumes ``amount`` is always a valid number and ``currency`` is always a valid ISO 4217 code supported by ``Intl.NumberFormat``. Invalid or unexpected inputs can throw at runtime or produce misleading output.

Fix These First

- 1 ``new Intl.NumberFormat('en-US', { style: 'currency', currency })`` will throw a ``RangeError`` if ``currency`` is invalid, empty, lowercase in some environments without normalization, or unsupported by the runtime.
- 2 The AI assumed ``currency`` is always a valid 3-letter ISO 4217 currency code such as ``USD`` and is supported by the runtime's ``Intl`` implementation.

What the AI Assumed

- HIGH The AI assumed ``currency`` is always a valid 3-letter ISO 4217 currency code such as ``USD`` and is supported by the runtime's ``Intl`` implementation.
- MEDIUM The AI assumed ``amount`` is always a finite numeric value that ``Intl.NumberFormat.prototype.format`` can safely format.
- MEDIUM The AI assumed the locale should always be hardcoded to ``en-US``, regardless of the user's language or region.
- LOW The AI assumed callers want JavaScript coercion behavior for ``amount``, including formatting numeric strings like ``123.45`` without explicit validation.

What Could Break

- HIGH ``new Intl.NumberFormat('en-US', { style: 'currency', currency })`` will throw a ``RangeError`` if ``currency`` is invalid, empty, lowercase in some environments without normalization, or unsupported by the runtime.
- MEDIUM If ``amount`` is ``NaN``, ``Infinity``, ``undefined``, or a non-numeric value, the output may be incorrect (``$NaN``) or inconsistent with business expectations instead of failing clearly.

MEDIUM

Hardcoding ``en-US`` can display the wrong separators, symbol placement, and formatting conventions for non-US users, which is risky in internationalized applications.

MEDIUM

If this function is used in financial workflows, relying on floating-point numbers without validation can introduce rounding surprises before formatting.

Verify Checklist

- [] Verify whether callers may pass non-number values such as ``null``, ``undefined``, numeric strings, ``NaN``, or ``Infinity`` into ``amount``.
- [] Verify that every ``currency`` value passed to ``formatCurrency`` is a valid ISO 4217 code and matches what your runtime's ``Intl.NumberFormat`` supports.
- [] Verify whether the application should use a caller-provided locale instead of always forcing ``en-US``.
- [] Verify whether financial values are already rounded correctly before formatting, especially if ``amount`` is derived from floating-point arithmetic.
- [] Verify expected behavior for invalid input: throw an error, return a fallback string, or log and continue.

Suggestion Cards

No input validation for ``amount``

Why it matters:

Unexpected values like ``undefined``, ``NaN``, or ``Infinity`` can produce invalid output or hide data issues until production.

Fix:

Convert ``amount`` explicitly with ``Number(amount)`` and reject non-finite values with ``Number.isFinite``.

No validation or normalization for ``currency``

Why it matters:

An invalid currency code causes ``Intl.NumberFormat`` to throw at runtime, which can break rendering paths.

Fix:

Normalize with ``String(currency).trim().toUpperCase()`` and validate against ``^[A-Z]{3}$`` before constructing ``Intl.NumberFormat``.

Locale is hardcoded to ``en-US``

Why it matters:

Users in other regions may see incorrect currency formatting conventions, reducing usability and trust.

Fix:

Accept ``locale`` as a parameter or derive it from application settings instead of hardcoding it.

No explicit error handling around `Intl.NumberFormat`

Why it matters:

Unsupported locales or currencies can throw exceptions that are harder to diagnose if left uncaught.

Fix:

Wrap formatter creation and formatting in a `try/catch` and throw a clearer error with the actual `locale` and `currency` values.